

AI ExamGuard

Final Defense – Speaking Script and Anticipated Questions

An AI-Monitoring Online Examination and Proctoring System for Arellano University (ETEEAP)

Companion document for the panel talking points: Introduction (background, statement of the problem, objectives) • Methodology (diagrams, flowcharts, how it was developed) • Results and Discussion (qualitative interpretation, response from beneficiaries). Part B covers objectives, related literature, operational terms, data-processing techniques, tools, methodology, and a machine-learning deep dive (dataset sources, training technique, model explanation).

Every figure quoted here is the number recorded in Chapters 1–5 and in the Model Training Methodology document. Where a result is in-sample, proxy-based, or has a known ceiling, the answer states it — the thesis reports them the same way, and that is the defensible position.

PART A – Speaking Script

Timing target ~9–11 minutes. Say it in your own words; the phrasing below is a floor, not a script to read verbatim. Bracketed cues point to the slide/figure to show.

1. Introduction

Background of the study. Arellano University, through its ETEEAP program, delivers much of its instruction and assessment online to working students. Examinations are still the main way learning is measured, so the credibility of the results depends on the exam being conducted fairly. In a physical room the instructor provides that assurance through direct supervision. Online, that assurance is largely gone: the instructor cannot watch every student, webcams show only a narrow view, and LMS platforms and Google Forms are built to deliver questions and collect answers, not to verify how the answers were produced.

The problem has also changed shape. Traditional remote cheating needed prepared notes or another person. Today an AI chatbot in a second browser tab answers almost any question instantly, with no preparation and no accomplice, and it leaves no visible sign on the student's face or desk. Commercial proctoring services exist but bill per student in foreign currency, stream continuous video and audio to outside servers, and do not disclose how their detection works — which is difficult for a Philippine institution to sustain and to defend under the Data Privacy Act.

Statement of the problem. Online examinations make academic integrity hard to maintain because instructors cannot continuously observe every examinee. The recurring concerns are unauthorized persons, mobile phones, the examinee leaving the camera view, and prolonged or repeated head movement away from the screen. Manual monitoring is inconsistent and does not scale. The study asks how image and video data can be collected, annotated, and preprocessed for a detection model; how YOLOv8 and OpenCV can detect those four observable events; what precision, recall, and F1-score the model achieves; how the full system can be designed with management, verification, real-time monitoring, evidence capture, alerts, and reporting; and what technology stack can deliver it within the study period.

Objectives. The general objective is to design, develop, and evaluate AI ExamGuard. Specifically: (1) collect, annotate, and preprocess data representing normal and suspicious exam conditions for training, validation, and testing; (2) develop a detection model using YOLOv8 and OpenCV for multiple persons, mobile phone use, absence from the camera, and unusual or repeated head movement; (3) measure its performance in precision, recall, and F1-score; (4) design and develop the full system — user and exam management, identity and camera verification, real-time AI monitoring, suspicious-event detection and logging, automated alerts, and monitoring reports with time-stamped evidence; and (5) implement it within the study period using YOLOv8, OpenCV, Python, a web framework, and a database system.

2. Methodology

Research design. The study is descriptive and developmental. The descriptive part recorded, through interviews and observation, how online exams are given now and where supervision breaks

down, and later reported how the finished system performed. The developmental part built the system itself, from requirements through design, dataset preparation, model training and tuning, implementation of the server, web app, and extension, and finally testing and improvement.

Development model. The agile incremental model was used: the project was divided into small features, each going through its own plan–analyse–design–build–test–review cycle. This was necessary, not just convenient — several requirements could only be decided after working code existed. The detection thresholds, the second phone check around the hands, the three-check corroboration rule, the rule for discarding face-shaped false detections, and the one-miss forgiveness in the head-down streak were all added or changed because of test results, not planned upfront.

Diagrams. [show Figures 2–9] The system was documented before coding: a block diagram of the three layers (client, application, data); a context diagram showing the three external users and every data flow crossing the system boundary; a data flow diagram exploding that single process into six numbered processes and six data stores; an entity-relationship diagram for the database; and screen prototypes drawn in Figma. Four procedure flowcharts describe run-time behaviour: identity checking and head position (Figure 5), prohibited-object and room monitoring (Figure 6), risk scoring (Figure 7), and the whole exam-and-appeal process (Figure 8).

How the detection was developed. Three trained components sit behind three flowcharts. (a) A phone-and-face detector: YOLOv8s, fine-tuned by transfer learning from COCO weights on a public cellphone image set re-organised into YOLO format plus hand-annotated real exam-room frames from the Michigan State University Online Exam Proctoring (OEP) database, split strictly by subject. (b) Per-student face identity verification: OpenCV's LBPH recognizer, trained at enrollment from at least three webcam samples per student. (c) A risk scorer: logistic regression over three vision signal counts, deliberately explainable. Head-pose for the head-down signal is not a trained model — it is solvePnP geometry (Perspective-n-Point) on the five facial points YuNet already returns. Almost no threshold was taken from a default; each was swept against labeled data, and several were corrected again after testing on a real webcam.

Testing and evaluation. Two rounds: the researchers self-tested the parts that need a reserved dataset or real-hardware measurement, then user-respondents exercised every module in their own role. Detector performance was measured on a frozen holdout used once per candidate model, and separately on live webcam frames. The finished system was then rated by evaluator-respondents against six ISO/IEC 25010 characteristics — functional suitability, performance efficiency, compatibility, interaction capability, reliability, and security — on a five-point scale, summarised with weighted mean, overall mean, and standard deviation.

3. Results and Discussion

Detection results. [show Table 6] Face matching at threshold 60 gave precision 0.950 and recall 0.873. The retrained phone detector, evaluated frame-by-frame on the 322-frame frozen holdout through the full deployed pipeline, gave precision 0.915, recall 0.987, F1 0.949. On 24 live frames of a phone actually held, the operating threshold of 0.35 flagged 20 (83%); the value that looked best offline flagged only 1, which is why the lower threshold was kept. Requiring a weak phone detection

to appear in three consecutive checks raised recall from 88.8% to 93.9% while false alarms rose only from 2.2% to 3.0%. The risk model scored ROC-AUC 0.797 under leave-one-subject-out cross-validation, with a 95% confidence interval of 0.666 to 0.941 — reported instead of the more flattering 0.879 single-split figure.

Two limits were found that no threshold removes: texture-based face recognition cannot distinguish a photograph of a person from the live person, and head-tilt angle overlaps heavily between a student using a phone and a student simply reading the screen. Both signals are therefore treated as hints for the instructor to review, never as proof, and the appeal process exists partly because of them.

Qualitative interpretation. [show Tables 7 and 8] User-respondents rated each module after using it in their own role; evaluator-respondents rated the finished system against the six ISO/IEC 25010 criteria after a demonstration and hands-on use. <State the overall mean from Table 8, the highest and lowest criteria, and whether the 3.51 target was met on every criterion.> Written comments were grouped by theme; the suggestions applied before the study ended are noted, and the rest are carried into the Chapter 5 recommendations.

Response from the beneficiaries. <Summarise what students, instructors, and administrators reported: students on the clarity of the exam screen and the pre-exam notice and on the appeal channel; instructors on the risk-sorted live monitor replacing a wall of video thumbnails and on the per-flag evidence; administrators on self-hosting, the 90-day evidence purge, and the audit log for Data Privacy Act compliance.>

Close. AI ExamGuard gives online exams at the school something the previous setup did not have: a reviewable record — a risk score, a timed list of detections, and a saved frame for each camera-based flag — produced by a system the school hosts and controls, whose thresholds and the testing behind them are published, and whose outputs can be appealed. It does not decide who cheated; it tells an instructor where to look.

PART B – Anticipated Questions and Model Answers

B1. Objectives and Scope

Your title says “detection model” (singular). How many models are there really?

Four trained components, plus one geometric method. (1) A YOLOv8s phone-and-face detector, fine-tuned for this study. (2) The stock YOLOv8 person detector, used unmodified for the multiple-persons count. (3) A per-student LBPH face recognizer, trained at enrollment. (4) A logistic-regression risk scorer that combines the vision signals. The head-down signal is not a model at all — it is Perspective-n-Point geometry (solvePnP) on five facial points. “Detection model” in the objectives is the umbrella term for this ensemble; Chapter 4 reports each part separately because they behave differently.

Objective 2 lists “unusual or repeated head movement.” What exactly did you implement?

Prolonged head-down: the head-pose pitch estimate staying below a set angle for a set duration, with one interrupting reading forgiven before the streak resets. It is a proxy for covert phone use below the camera line. We implemented the sustained-downward case, not every conceivable head movement, because that is the case with a plausible cheating interpretation and a measurable duration. Testing showed the angle alone is weak (about 50% precision), so it is paired with the duration requirement, the on-screen question is captured with it, and it is sent to the instructor to judge rather than penalised automatically.

Did you meet all the objectives?

Objectives 1, 2, 4, and 5 were met: the data was collected, annotated, and preprocessed; the four events are detected; the full system with all listed features was built and deployed on school-hosted hardware via Docker and a Cloudflare Tunnel; the stack is Python 3.12/FastAPI, PostgreSQL 18, React 19, OpenCV, YOLOv8, scikit-learn. Objective 3 was met and reported honestly — precision, recall, and F1 were measured, and where a signal has a ceiling (head tilt, photo spoofing) that is stated as a finding rather than hidden.

B2. Related Literature

Which single study is your foundation, and how do you differ from it?

Atoum et al. (2017), “Automated Online Exam Proctoring.” They split detection into six components combined by a classifier over time, collected data from 24 subjects asked to cheat, and released that dataset — which is our main labeled source. We keep their core design (separate detectors, combined over time) and their stated position that perfect automatic detection is the wrong goal and high-probability cases should go to a human. We differ in three ways: still frames every ~5 seconds instead of continuous video; published thresholds and the testing behind them; and self-hosting on the school’s own machine.

What does the Fraud Triangle contribute — isn’t it just a business-fraud theory?

It bounds what the study can claim. Cressey’s triangle — pressure, opportunity, rationalization — is widely applied to academic dishonesty. Of the three, software can only reduce *opportunity*. That is why the thesis does not claim to stop cheating, only to reduce the chance of doing it unseen and to leave a record. It also explains an observation from our background research: when students see rules going unenforced online, cheating starts to look normal, which strengthens rationalization — so having any credible enforcement matters beyond the individual catch.

Why YuNet for face detection instead of a more accurate modern detector?

Wu, Peng, and Yu (2023): YuNet reaches 81.1% mAP on WIDER FACE hard with about 75,856 parameters and runs in ~1.6 ms per frame at 320×320 on a desktop CPU. We have no GPU in production and must run it every ~5 seconds for many students. It also stays reliable when the head is turned or lighting is poor, unlike the Haar cascade we used before, which caused false face-lost flags on any brief head turn. And it returns five facial landmarks with the box, which we feed straight into head-pose — no extra model needed.

Why LBPH for recognition when deep face embeddings are the standard?

Ahonen et al. (2006). LBPH needs little compute, tolerates lighting change, and trains from only a few enrollment photos per student — all three matter when hundreds of students enroll from a handful of webcam shots and must be matched without a GPU. Its weakness is documented and we confirmed it: texture-only means no liveness signal, so a printed photo produces a texture it cannot distinguish from a live face. That is stated as a limitation and is why identity is a hint plus an appeal path, not a lock.

What gap in the literature does your study fill?

Three. (1) The AI-chatbot problem: visual detection cannot see a second browser tab, and browser-side monitoring for it is comparatively under-studied — we add an extension that watches a published list of chatbot and search domains. (2) Photo-spoofing of lightweight face recognition is admitted in principle but rarely measured in a working system — we measure it live and report the negative result that frame-differencing does not fix a hand-held photo. (3) The constraints of a self-hosting institution — no GPU, limited bandwidth, its own hardware, Philippine data-privacy law — are mostly absent from a literature dominated by commercial services.

B3. Operational Terms the Panel May Probe

Term	How it is used in this study
Violation	One recorded detection of any of the 12 watched types, saved with type, time, session, optional detail, and (camera)
Evidence Frame	The single webcam still saved at the moment a camera-based violation fires, so the instructor deciding and the st
Corroboration Window	The last three object checks in a session. A low-confidence phone detection becomes a violation only if it appear
Frozen Holdout	A stratified 15% slice (322 frames) removed before any training and used once per candidate model. Never used
Head-Down Streak	Consecutive face checks with pitch below the set angle; the violation fires after a set duration, with a limited num
Pose-Guided Re-Check	Second phase of phone detection: a pose model locates the wrists and the detector re-runs at a lower threshold

Liveness Check	Compares each face crop with the previous one; a run of near-identical crops is flagged as a possible photo held
Risk Score / Risk Band	One value 0–100 per session (trained model for camera signals + fixed weights for browser signals, overturned a
Multi-Tenancy	One installation serving several schools with fully separated data; no user of one school can read or change anot
Exam Accommodation	An administrator exception letting one student skip the face check or the object check, or get extra time; enforced

B4. Data-Processing Techniques

Walk us through how a raw video becomes training data.

Six steps. (1) **Frame extraction** — only the webcam track of each OEP subject is decoded, at one frame per second, which is also the anonymization boundary: the extracted frames carry a subject code, not the source identity. (2) **Prioritization** — ~9,700 raw frames is too many to label, so frames were prioritized (phone-timestamp windows first, then a spread across subjects and conditions) into a manageable batch. (3) **Machine draft annotation** — the existing detectors draw draft boxes at a deliberately loose threshold so a reviewer corrects rather than draws from scratch. (4) **Human correction** in LabelImg — every draft box reviewed and fixed by hand; this is the ground-truth step, because training the next model on the current model's own guesses would just teach it the current blind spots. (5) **Label-defect remediation** — a per-subject accuracy breakdown flagged subjects far below average; one class of mislabeled boxes (an eye-tracker device read as a phone) was corrected across all subjects. (6) **Preprocessing at train/inference** — letterbox resize to 416×416, pixel scaling, and for training only, YOLO's standard augmentation (mosaic, HSV jitter, flips).

What preprocessing does a face frame go through for recognition?

Identical at enrollment and verification: YuNet detects the face, the largest box is cropped, converted to grayscale, and resized to a fixed size before LBPH. Frames where YuNet finds no face are dropped; enrollment needs at least three usable samples or it refuses and tells the student to retry rather than saving a weak profile. The original enrollment photos are not kept — only the trained model.

How are the risk-model features built from raw detections?

Detections are counted inside the scoring window, per type. The three features are `face_lost_count`, `phone_detected_count`, and `multiple_people_count` — the three signals with a visual correlate in the OEP ground truth. Browser signals have no correlate in any recorded proctoring dataset, so they keep expert-assigned weights instead of being forced into the trained model, and that split is stated openly.

Isn't machine-drafted annotation circular — the model labeling its own training data?

It would be if the drafts were used as-is. They are not. The draft only positions a box for a human to accept, move, or delete in LabelImg; the corrected box is the label. The loose draft threshold is chosen so misses cost more than false positives (drawing from scratch is slower than one delete). The per-subject audit and the defect remediation exist precisely to catch cases where uncorrected drafts slipped through.

B5. Tools Used

Layer / purpose	Tools
Backend	Python 3.12, FastAPI, SQLAlchemy + Alembic, PostgreSQL 18, JWT auth, bcrypt via passlib, SlowAPI rate limit
Frontend	JavaScript, React 19, Vite, Tailwind CSS 4, React Router 7, Recharts, Axios; browser MediaCapture / Fullscreen
Extension	Chrome Extensions Manifest V3 (event-driven background service worker; site-specific permissions only)
Computer vision	OpenCV (contrib) — YuNet via the DNN module from ONNX, LBPH recognizer, solvePnP; Ultralytics YOLOv8
Machine learning	scikit-learn (logistic regression, cross-validation, metrics), joblib (model persistence), pandas (feature assembly)
Dev / deploy	Git + GitHub, GitHub Actions CI (incl. a check that contrib OpenCV was not silently replaced), pytest + coverage

Why FastAPI and PostgreSQL specifically?

FastAPI validates incoming data and generates its own API documentation from Python type hints, which cuts hand-written validation code. PostgreSQL gives durable storage, enforced foreign keys, and correct timestamp handling — the logging and 90-day retention features depend on all three. Alembic keeps a numbered migration history so the same schema is recreated in dev, CI, and production.

Why a browser extension in addition to the exam page — can't the page do it alone?

No. Browser security prevents a web page from seeing activity in other tabs. The exam page detects only what concerns itself — losing focus, leaving fullscreen, copy/paste, right-click. Detecting a visit to a chatbot needs an extension with declared permissions. It requests access only to the specific listed sites, never reads page content or keystrokes, observes navigation without blocking it, and does nothing outside an active exam.

B6. Methodology and Development Process

Why agile incremental and not the classic waterfall / SDLC?

Because several requirements were genuinely unknowable up front — they depended on measurements that only exist once code runs. The phone confidence threshold, the need for the hand-region second pass, the three-check rule, the face-shaped-box discard rule, and the one-miss forgiveness were all added or revised from test results. Waterfall assumes requirements are fixed at the start; ours were not, so a model that lets requirements change between increments was necessary, not merely preferred.

How did you validate the system, concretely?

Two rounds. Researcher self-testing covered what normal use does not reach: invalid and extreme inputs, cross-role and cross-school access attempts, interruption and recovery, and the model tests that need the reserved dataset. Then user-respondents ran every module in their own role, in usage order, so a fault in an earlier module surfaced before the modules that depend on it. The backend also carries automated tests run on every commit via GitHub Actions.

Your sample — how many respondents and how were they chosen?

<State the Table 5 numbers: population, respondents per group (students, instructors, administrators, IT-practitioner technical evaluators), and percentages.> Students are the largest share because the monitoring affects them most directly and their experience of the exam screen weighs most on the interaction-capability rating. <State the sampling method used — e.g. purposive for the role groups.>

What statistical treatment did you apply to the evaluation data?

Weighted mean per item and per ISO/IEC 25010 criterion, overall mean across the six criteria, and standard deviation to show rating agreement. Open-ended comments were grouped by theme, not computed. Results were interpreted with the Table 4 scale (4.51–5.00 Excellent, 3.51–4.50 Very Good, and so on). The Chapter 1 target was a weighted mean of at least 3.51 on every criterion and overall.

B7. Machine-Learning Deep Dive

What are your dataset sources, exactly, and are they permitted for this use?

Three. (1) The **Michigan State University Online Exam Proctoring (OEP) database** from Atoum et al. (2017) — real exam-room webcam video of subjects performing scripted cheating, released for research; it is the source of the annotated proctoring frames and the only data with cheat-type ground truth for the risk model. (2) A **public cellphone image dataset** (~22.9k images) re-organised into YOLO train/val/test format — general phone pictures to give the detector volume and variety. (3) **Locally captured webcam frames** on real hardware, taken by the researchers for live threshold validation and for the frozen-holdout's live-condition frames. Enrollment face photos from participating test users are used only to train their own recognizer and are not retained.

Why fine-tune a separate phone model instead of retraining the one YOLOv8 you already have?

Retraining a detector on a phone-only dataset rebuilds its output head to know only that one class, which silently deletes the *person* class the multiple-people check needs. So the system keeps two detectors: the untouched COCO YOLOv8 for counting people, and a separately fine-tuned YOLOv8s that outputs phone and face. They run as two passes. This is documented as a design decision with a reason, not an oversight.

Describe the training technique for the phone detector.

Transfer learning. Start from COCO-pretrained YOLOv8s weights and fine-tune all layers — no frozen backbone — on the combined dataset (public phone images + subject-split OEP frames). Input 416×416, batch 8, SGD with Ultralytics' default schedule, mosaic and HSV and flip augmentation, early-stopping patience of 8 epochs, on a single local CUDA GPU. The data YAML lists the phone set's own train/val plus the OEP split; the OEP split is **by subject, never by frame**, because frames from one subject share a face, room, and camera and a per-frame split would leak near-duplicates into validation and inflate the score.

Explain each model in one breath, as if to a non-specialist.

Phone-and-face detector (YOLOv8s). A single-stage network: one pass over the image predicts boxes and classes together, so cost does not grow with the number of regions checked — that is what makes repeated checks on ordinary hardware feasible. Fine-tuned so it recognises phones held at an angle, back-first, or partly hidden by the hand, which the generic COCO phone class handles poorly.

Person detector (stock YOLOv8). Unmodified COCO model, used only to count people in frame; more than one triggers the multiple-persons violation.

Face identity verification (LBPH). Divides the face into regions, histograms the local binary texture pattern in each, concatenates them, and compares by distance. Trained per student at enrollment from ≥ 3 samples. A distance above the threshold is an identity mismatch.

Risk scorer (logistic regression). Predicts the probability of cheating as a weighted sum of three detection counts. Chosen for explainability — each learned coefficient says exactly how much one more occurrence of a signal moves the probability, which an instructor can be shown and a student can contest.

Head-pose (not a model). solvePnP / Perspective-n-Point: fit a generic 3-D face model to the five 2-D facial points and read off the pitch angle. A geometric estimate of where the head points — not where the eyes look.

How do you compute precision, recall, and F1 — per box or per frame?

Per **frame**. A frame is a true positive if it contains a real phone and the pipeline reports a phone anywhere in it; a false positive if it reports a phone in a phone-free frame; a false negative if it misses a real one. This matches what production actually does — log a violation when a phone is detected — which has no notion of box location. Precision = $TP / (TP + FP)$, recall = $TP / (TP + FN)$, F1 = $2 \cdot P \cdot R / (P + R)$. The harmonic mean is used so a model cannot get a good score by being strong on one and weak on the other.

What are the headline numbers, and which is the honest one?

Phone detector on the 322-frame frozen holdout, full deployed pipeline, threshold 0.35: **precision 0.915, recall 0.987, F1 0.949** (TP 75, FP 7, FN 1 of 76 phone-positive frames). Read plainly — of every 100 frames flagged for a phone about 92 really have one, and about 99 of every 100 phone frames are caught; the residual error is on the precision side. Face verification at threshold 60: precision 0.950, recall 0.873. Risk model: ROC-AUC **0.797** under leave-one-subject-out cross-validation, 95% CI 0.666–0.941 — the honest figure; an earlier 0.879 came from a single split with only two held-out subjects.

Your earlier draft said F1 0.854. Why did it change?

A label defect in the holdout, found 2026-09-08 and corrected. The fairness audit had already removed 579 boxes where the OEP rig's own second camera — a chrome camcorder on a stand beside the subject's head — was annotated as a phone, but that fix was applied to the training batch, and the frozen holdout sits physically outside it so no training script can reach it, so the correction never propagated in. It surfaced while diagnosing why three retrains all appeared to regress: the misses were not spread across conditions but absolutely concentrated — one 16-frame batch scored 16/16 missed while the same subject's other frames scored 30/30. Those 16 frames contain no phone. 41 boxes were removed after frame-by-frame visual verification, moving 20 frames to verified negative. Say plainly that the correction cut both ways: recall rose $0.792 \rightarrow 0.987$, and precision *fell* $0.927 \rightarrow 0.915$, because one frame had been scoring a free true positive for a detection on the camera device.

Why is the risk-model confidence interval so wide (0.67 to 0.94)?

Leave-one-subject-out on a small number of usable subjects. Only 11 OEP subjects had usable labels for camera-visible cheat types; refitting once per held-out subject on that few gives a genuinely wide interval. It is reported as-is rather than hidden, because it honestly says the accuracy on a

brand-new student is only known within wide bounds — which is also why the output is a band shown to an instructor, not an automatic penalty.

How did you tune the thresholds, and did any offline choice fail live?

Each threshold was swept across a range against labeled data, with the pipeline run once per frame and the sweep done analytically from the recorded scores. Yes — one failed live and it is kept in the thesis as a load-bearing finding: the sweep recommended raising the phone threshold from 0.35 to about 0.70 on frozen-holdout F1. On a live webcam with a phone actually held, 0.70 caught 1 of 24 frames and 0.35 caught 20. The holdout's phone frames were all one dim lighting condition. The threshold stayed at 0.35, and the rule 'an offline-optimal threshold is not shippable without a live check' became explicit.

What is the frozen holdout and why does it matter?

A stratified 15% slice (322 frames, fixed seed) carved from every training-eligible source *before* any model was trained. Rules written into the scripts: never train on it, never use it to pick thresholds, run it once per candidate model, and report whatever it says. It exists because the two validation subjects had been reused across several model-swap decisions, which is leakage — repeatedly consulting the same set to choose a model quietly fits to it.

Leave-one-subject-out — why that instead of a normal train/test split?

Because the question is 'how will this do on a student it has never seen,' and a random split puts frames from the same subject on both sides. LOSO refits the model N times, each time holding out one whole subject, tests on that subject, and pools the predictions. It is the honest estimate of per-new-student performance and it is what produced the wider, truthful 0.797.

Did you record any negative results?

Yes, on purpose — a methodology that only reports what worked is not a methodology. (1) Frame-difference liveness does not reliably catch a hand-held photo, because a hand shakes enough to mimic a live face. (2) Raising the phone threshold on offline evidence crashed live recall. (3) A second, independent head-pitch signal was tried and reverted because it worsened F1 on real labeled data. Each is written up where the relevant decision is made.

Biggest ML limitations, stated plainly?

The proctoring data is a small number of people *asked* to cheat, so the positive rate (~71%) is far above any real class and the behaviour may be more obvious than real covert cheating; the risk model's output is rescaled so a clean session scores 0.

The datasets carry no demographic information, so a proper fairness study is impossible with public data — we can compare per-subject and per-batch accuracy but cannot attribute a gap to skin tone, lighting, or camera. This is the top recommendation for future work.

Texture-based recognition has no liveness signal; head-pitch overlaps too much between phone use and screen reading. Both are ceilings of the method, not tuning problems, which is why evidence capture, appeals, and the audit log are core, not optional.

CPU-only inference caps how often checks run and how many students one machine serves; per-session state is in memory and is lost on restart.

If a newer figure is quoted at you (dataset later expanded, etc.) — how to answer?

Answer with the numbers written in the thesis of record: phone precision 0.915 / recall 0.987 / F1 0.949 on the corrected frozen holdout, risk-model LOSO AUC 0.797 (CI 0.666–0.941). Note Chapter 4's Table 6 does not cite the frozen-holdout figures at all — it reports the live-hardware tests, which the label correction does not touch — so nothing in Chapters 1–5 changed. If you have since expanded the risk dataset or retrained a detector, give the newer figure as *additional* continuing work. Never quote a number you cannot point to in a file.

B8. One-Line Rebuttals to Keep Ready

- **“83% live phone recall is low.”** — It is a single-frame number; the three-check rule and the pose-guided re-check raise the session-level figure, and a missed frame is re-checked ~5 seconds later.
- **“The head-down check is unreliable.”** — Correct, and we say so; it is a review hint with a duration gate and captured question context, never a standalone proof.
- **“Face recognition can be fooled by a photo.”** — Yes; that is a documented ceiling of texture-based recognition. It is mitigated by review and appeal, and blink-based liveness is the recommended next step.
- **“Why not use a commercial proctoring API?”** — Recurring per-student cost in foreign currency, student data sent abroad, and undisclosed detection logic that cannot be defended under the Data Privacy Act.
- **“Only 11–24 subjects.”** — It is the largest public dataset with real exam-room video and cheat-type labels; the wide confidence interval we report is the honest consequence, not a hidden flaw.
- **“The system accuses students.”** — It does not. It outputs a risk band and the evidence behind it; the finding and any penalty stay with the instructor and the existing disciplinary process.

Prepared as a study aid from the project's own documentation. Fill the <bracketed> evaluation figures from Tables 5, 7, and 8 before the defense.